# End-to-End Data Management Pipeline for a Recommendation System

**Student:** Syed Amaan Hasan (individual project) **ID:** 2025AG05941
**Course:** Data Management for Machine Learning (DM4ML), Assignment 2 **Date:** 11 November 2026
**Repository:** https://github.com/hasansyedamaan/dmml_assignment2 (branch `main`)

## 1. Introduction and objective

Raw interaction data contains missing values, duplicates, wrong types, invalid ratings and timestamps and orphan references. A recommender trained on it can silently learn from corrupt rows or leak future behaviour. This project builds a modular, reproducible pipeline on **MovieLens 100K** (943 users, 1,682 movies, 100,000 ratings) covering ingestion, validation, preparation, transformation, feature management, model training, evaluation and orchestration. The emphasis is on data management and reproducibility rather than on maximising accuracy.

## 2. Architecture, design decisions and tools

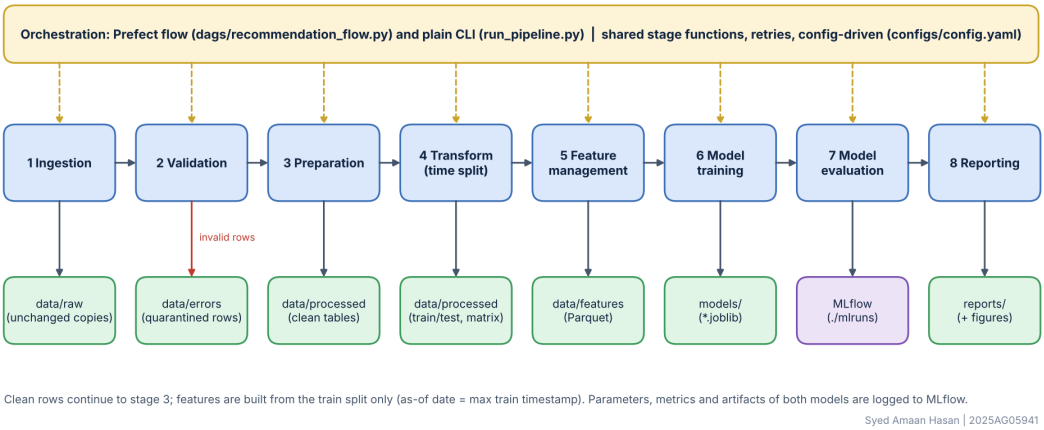



*Figure 1: Pipeline architecture, data stores, tracking and orchestration.*

- **One set of stage functions** (`src/pipeline.py`) is called by both the plain CLI and the Prefect flow, so the two entry points cannot drift apart.
- **Quarantine, not silent dropping:** invalid rows go to `data/errors/` with the failed rule; only clean rows reach later stages.
- **Leakage control:** the split happens before feature building, and features use the training split only, with an explicit as-of date.
- **Config-driven:** paths, seed (42), K, split rule and model parameters live in `configs/config.yaml`; storage is Parquet (typed, compact, inspectable).

| Tool | Role | Tool | Role |
|---|---|---|---|
| pandas, NumPy, SciPy | Tabular processing, sparse matrix | MLflow | Params, metrics, artifacts (local store) |
| Pandera + rule checks | Schema gate and row-level validation | Prefect | Orchestration, retries, dependencies |
| PyArrow / Parquet | Processed and feature storage | scikit-learn | TruncatedSVD (implicit/Surprise had no compatible wheels) |
| pytest | 21 unit tests | matplotlib, ReportLab | Figures and PDF |

## 3. Data validation and quarantine

Validation (`src/validation/validators.py`) combines row-level rules that record every failed rule per row with a Pandera schema gate on the surviving rows. Checks: missing values, duplicates, incorrect data types, ratings outside [1, 5], unparseable/future/too-old timestamps, invalid IDs, and referential integrity (orphan `user_id` / `item_id`). To demonstrate the checks, a documented fault-injection option adds known bad records to a *copy* of the raw data. All 14 injected records were detected and written to `data/errors/`; full counts are in `reports/validation_report.md`.

| Dataset | Rows in | Valid | Rejected | Checks that fired (rows flagged) |
|---|---|---|---|---|
| Users | 945 | 943 | 2 | missing value (1), duplicate (1) |
| Items | 1,684 | 1,682 | 2 | missing value (1), duplicate (1) |
| Interactions | 100,001 | 99,991 | 10 | missing (1), duplicate (1), dtype (2), rating (2), timestamp (3), orphan user (1), orphan item (1) |

*A row can fail several rules, so flagged counts exceed rejected rows. Pandera passed on all three clean tables. The injection overwrites nine existing interactions in the copy, so the fault-injection run uses 99,991 rows instead of 100,000 (see the confirmation run in Section 6).*

## 4. Preparation, transformation and time-based split

- **Cleaning:** rows missing IDs/ratings/timestamps dropped; missing age → median, gender → U, occupation/ZIP → unknown, title → Untitled; duplicates keep first; timestamps → UTC; genres → list + primary genre.
- **Split** (`per_user_last_fraction`): each user's chronologically last 20% (at least 1) forms the test set. Train 79,614 / test 20,377 interactions; 943 test users, 0 cold-start users, 71 cold-start test items (88 rows) that cannot be recommended and count as misses. A global cutoff was rejected because it creates many cold-start users on this dataset.
- **Derived datasets** (train-only, `data/processed/`): user activity summary (943 rows), item popularity (1,611 rows) and a sparse user-item matrix (943 × 1,611, 79,614 non-zeros) with ID index files.

## 5. Feature management

A reusable `FeatureStore` (`src/features/feature_store.py`) builds features from the training split only (as-of date = maximum train timestamp, 1998-04-22 23:10:38 UTC) and serves them through `get_user_features`, `get_item_features` and `get_interaction_features` with documented defaults for unseen entities, so training and prediction share one code path. The 18-entry feature catalog is generated programmatically from the code registry (`reports/feature_catalog.md/.csv`) so documentation cannot drift.

| Entity | Features |
|---|---|
| User | avg_rating, activity_count, preferred_category, days_since_last_interaction, rating_std, distinct_items_count |
| Item | popularity, avg_rating, rating_count, category, genre_* (one-hot), item_age_years |
| Interaction | rating, timestamp, hour, day_of_week, user_item_genre_match, recency_rank |

## 6. Models and evaluation

Both models share the interface `fit / recommend / predict`. **Popularity baseline:** ranks unseen items by training interaction count (item mean rating for rating prediction). **Matrix factorization:** scikit-learn *TruncatedSVD* on the sparse rating matrix (32 components, 12 iterations, seed 42) for ranking, plus a bias-aware residual SVD, $\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i$ (clipped to [1, 5]), for rating prediction. Both runs are logged in MLflow (experiment `dm4ml-recommendation-pipeline`) with parameters, metrics and artifacts. An item is *relevant* if its test rating ≥ 4; ranking metrics average over the 908 of 943 test users with at least one relevant item. Coverage = unique recommended items / 1,611 training items; Diversity = mean pairwise cosine distance between genre vectors in each list.

| Metric | Popularity K=10 | Matrix fact. K=10 | Popularity K=5 | Matrix fact. K=5 |
|---|---|---|---|---|
| Precision@K | 0.0754 | **0.1320** | 0.0769 | **0.1566** |
| Recall@K | 0.0749 | **0.1543** | 0.0402 | **0.0941** |
| NDCG@K | 0.0943 | **0.1763** | 0.0880 | **0.1746** |
| Coverage@K | 0.0447 | **0.2862** | 0.0273 | **0.2228** |
| Diversity@K | **0.7478** | 0.7128 | **0.7812** | 0.6970 |
| RMSE / MAE (rating prediction) | 1.0735 / 0.8587 | **1.0082 / 0.7915** | – | – |

*Table 1: Model comparison on the time-based test set (best value in bold).*

At K = 10 the personalized model improves Precision +75%, Recall +106%, NDCG +87%, catalog coverage 0.045 → 0.286 (6.4×), RMSE −6.1% and MAE −7.8%. Diversity falls 4.7%, as expected when lists are tailored to each user's genres. Results come from one run with fixed hyperparameters and no significance test, so they demonstrate the pipeline rather than a leaderboard result.

A confirmation run through the CLI on the unmodified data (no fault injection, 100,000 interactions, same 943 evaluated users) gave matrix factorization Precision@10 0.1315, Recall@10 0.1538, NDCG@10 0.1758, Coverage 0.2868, Diversity 0.7127, RMSE 1.0082 and MAE 0.7914, against 0.0754, 0.0749, 0.0943, 0.0447, 0.7479, 1.0734 and 0.8587 for popularity. The model ranking is unchanged, and the small differences from Table 1 (at most 0.0006) are consistent with the nine-row difference in input data.

## 7. Orchestration and reproducibility

The Prefect flow (`dags/recommendation_flow.py`) wraps each stage as a task with two retries and runs them in dependency order: ingest → validate → prepare → transform → features → train → evaluate → report. Figure 2 shows a successful end-to-end run (`python dags/recommendation_flow.py --inject-faults --skip-download`): all eight stages completed, validation rejected the 10 faulty interactions, both MLflow runs were logged and the flow finished in the completed state. The log is saved in `reports/run_log.txt`. The same stage functions also run through the plain CLI (`python run_pipeline.py`), which produced the confirmation run in Section 6.

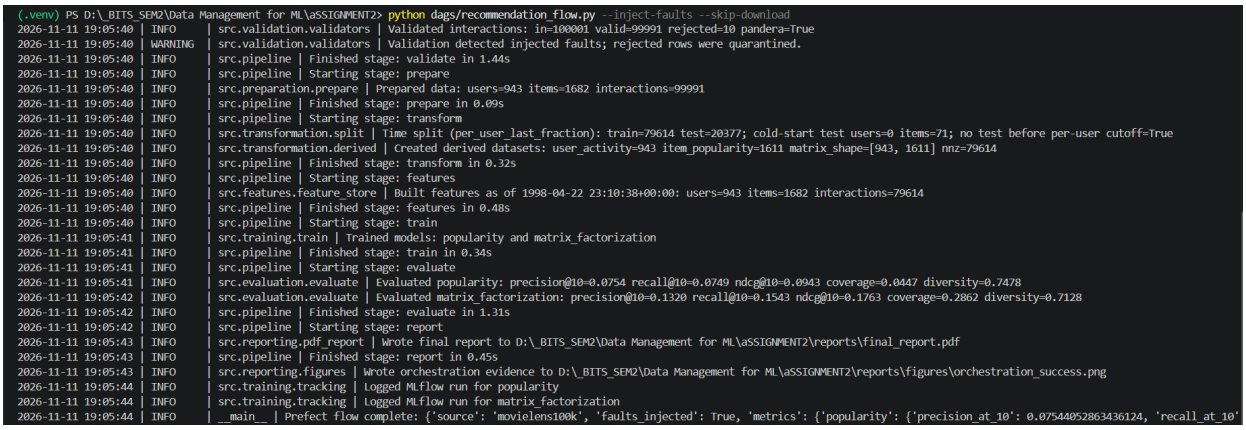



*Figure 2: Screenshot of a successful Prefect flow run (validation through flow completion).*

**Reproducibility.** The fault-injection flow was run on 10 and 11 November 2026 and the metrics matched to four decimal places (for example matrix-factorization Precision@10 0.1320, Recall@10 0.1543, NDCG@10 0.1763, Coverage 0.2862, RMSE 1.0082). This rests on a fixed seed (42), config-driven paths, pinned dependencies in `requirements.txt`, a deterministic time-based split and 21 passing pytest tests (validation, split, metrics, features, ingestion, models). Setup and run commands are in `README.md`.

## 8. Limitations

- **Features are not yet consumed by the models:** both recommenders train on the interaction matrix; the feature store is validated by tests and ready for a feature-aware or hybrid model.
- **Split:** the per-user temporal split keeps every user warm, but test periods differ across users and 71 test items are unseen in training.
- **Scope:** MovieLens 100K is small and explicit-feedback only; hyperparameters were not tuned; fault injection is synthetic; features are batch-only (no online serving).

## References

[1] F. M. Harper and J. A. Konstan, “The MovieLens Datasets: History and Context,” ACM TiiS, vol. 5, no. 4, 2015. [2] GroupLens Research, MovieLens 100K, https://files.grouplens.org/datasets/movielens/ml-100k.zip. [3] MLflow Tracking, https://mlflow.org/docs/latest/tracking.html. [4] Prefect, https://docs.prefect.io/. [5] Pandera, https://pandera.readthedocs.io/.